```
        this.#phoneNumber = phoneNumber;
        this.#since = since;
    }

    getName(){
        return this.#name;
    }

    getPhoneNumber(){
        return this.#phoneNumber;
    }

    setPhone(phoneNumber){
        if(phoneNumber == null)
            throw "Invalid phone number";
        this.#phoneNumber = phoneNumber;
    }

    getSince() {
        return this.#since;
    }

    toString() {
        return `${this.#name} ${this.#phoneNumber}
${this.#since}`
    }
}

export default User;
```

Though the setters are not really required in the context of *SearchService*, which is read-only, we offer *setPhone()* operation just as an illustration.

Developing the value objects and entities according to our design is that simple in ES. However, it is not the same when it comes to *UserRepository*. Our design experts an interface for the repository. Interfaces are really important and useful when we use languages like Java which are traditional and statically typed, but not in the case of ECMAScript which is a dynamic language. In fact, ECMAScript does not even offer a way to build the interfaces.

Hence, instead of deviating from the philosophy of the language, let us build the *UserRepository* without an interface. The repository connects to MySQL and executes SQL queries.

Let us import the classes from the domain layer and also the *mysql2/promise* module.

```
import mysql from 'mysql2/promise';
import User from '../domain/User.js';
import PhoneNumber from '../domain/PhoneNumber.js';
import Name from '../domain/Name.js';
```

Creating a configuration object for database connection is a good idea, as follows:

```
const config = {
    db: {
        host: process.env.DB_HOST || "localhost",
        user: process.env.DB_USER || "root",
        password: process.env.DB_PASSWORD || "adminadmin",
        database: process.env.DB_DATABASE || "glarimy",
    }
};
```

The deployment engineer can override the default value of the environment variables DB_HOST, DB_USER, DB_PASSWORD and DB_DATABASE at various levels that include docker, docker-compose and Kubernetes. This brings in a lot of flexibility without changing the code.

Finally, the *UserRepository* class offers just one method: *findByPhoneNumber()*. The method makes a connection with MySQL, passes the query for execution and converts the retrieved records into an array of *User* objects. Since Node follows the Reactor pattern, many of its I/O operations are async in nature. Consequently, the *findByPhoneNumber* is also async.

```
class UserRepository {
    async findByPhoneNumber(phoneNumber) {
        const connection = await mysql.
createConnection(config.db);
        const [records,] = await connection.
execute(`SELECT name, phone, since from ums_users where
phone=${phoneNumber}`);
        const users = new Array();
        for (let record of records)
            users.push(new User(new Name(record['name']), new
PhoneNumber(record['phone']), record['since']))
        return users;
    }
}

const repo = new UserRepository();
export default repo;
```

Before leaving the domain layer, observe what we are exporting from the *UserRepository* module. We are not exporting the class; instead, we are exporting an object of the *UserRepository*. This makes sure that there is only one instance of the repository in the run. This is the simplest way of making a class singleton in ECMAScript.